

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)**Department of Computer Science and Engineering****ASSESSMENT TOOL 2 – Coding Challenge**

Course Code:	DSA0405	Course Title:	FUNDAMENTALS OF DATA SCIENCE
CO Assessed:	CO5-AT2 Coding Challenge	Assessment:	ASSESSMENT TOOL 2 – Coding Challenge
Weightage:	40% of CIA	Total Marks:	100
CO4 Statement:	To understand the concept of supervised and unsupervised methods in machine learning		
Student Name:	V.Naga Nischay	Reg. No.:	192472255
Batch	2024	Date:	27-08-2026

Code of Conduct

I V.Naga Nischay (192472255) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Nischay

Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Marks
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly	
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts	
Analysis	20	In-depth	Good analysis	Basic analysis	Weak or no	

		analysis with strong reasoning and insights	with logical reasoning	with limited depth	analysis; lacks reasoning	
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided	
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand	

Code:

[12]

```

import numpy as np
import matplotlib.pyplot as plt

X = np.array([
    [1, 1],
    [2, 1],
    [3, 2],
    [4, 3],
    [5, 4],
    [6, 5],
    [7, 5],
    [8, 6],
    [9, 7],
    [10, 8]
], dtype=float)
y = np.array([
    0, 0, 0, 0, 0,
    1, 1, 1, 1, 1
], dtype=float)
X = np.c_[np.ones(X.shape[0]), X]
def sigmoid(z):
    return 1 / (1 + np.exp(-z))
def loss(X, y, theta):
    z = X @ theta
    p = sigmoid(z)
    p = np.clip(p, 1e-15, 1 - 1e-15)
    return -np.mean(
        y * np.log(p) +
        (1 - y) * np.log(1 - p)
    )
def gradient_descent(X, y, learning_rate=0.1,
                    max_iter=10000, tolerance=1e-6):
    m, n = X.shape
    theta = np.zeros(n)
    losses = []
    for iteration in range(max_iter):
        p = sigmoid(X @ theta)
        gradient = (X.T @ (p - y)) / m
        theta_new = theta - learning_rate * gradient
        current_loss = loss(X, y, theta_new)
        losses.append(current_loss)
        if np.linalg.norm(theta_new - theta) < tolerance:
            theta = theta_new
            break
        theta = theta_new
    return theta, losses, iteration + 1
def newton_method(X, y, max_iter=100,
                 tolerance=1e-6):
    m, n = X.shape
    theta = np.zeros(n)
    losses = []
    for iteration in range(max_iter):
        p = sigmoid(X @ theta)
        gradient = (X.T @ (p - y)) / m
        R = np.diag(p * (1 - p))
        H = (X.T @ R @ X) / m
        epsilon = 1e-6
        H = H + epsilon * np.eye(n)
        delta = np.linalg.solve(H, gradient)
        theta_new = theta - delta
        current_loss = loss(X, y, theta_new)
        losses.append(current_loss)
        if np.linalg.norm(theta_new - theta) < tolerance:
            theta = theta_new
            break
        theta = theta_new
    return theta, losses, iteration + 1

```

Output:

